Sales Forecasting using Time Series Analysis

Full-Stack Web Application Project Report & Presentation Guide

1. Project Abstract & Problem Statement

Businesses frequently face operational challenges in predicting future sales demand accurately. Manual sales forecasting and static spreadsheets are time-consuming, prone to human error, and fail to capture underlying time-series trends and seasonal patterns. Inaccurate predictions lead to severe consequences such as **stock shortages (lost revenue)** or **overstocking (high holding costs and financial waste)**.

This project delivers an end-to-end full-stack web-based solution that automates sales forecasting using **Time Series Analysis (ARIMA / SARIMA)**. Users can upload any sales CSV dataset, auto-detect dates and revenue columns, visualize historical trends, train forecasting models, evaluate accuracy metrics in Indian Rupees (■), and export projected forecasts.

2. Technical Architecture & Tech Stack

Layer	Technology / Tools	Purpose
Frontend UI	HTML5, CSS3, JavaScript (ES6+)	Responsive dark glassmorphism analytics dashboard
Data Charts	Chart.js	Interactive trend, backtest, and forecast visualization
Backend Framework	Python Flask (REST APIs)	Data ingestion, file handling, and API endpoints
Machine Learning	Statsmodels (ARIMA / SARIMA)	Time series model fitting, AIC optimization & prediction
Data Processing	Pandas, NumPy	CSV parsing, linear interpolation, frequency resampling
Model Evaluation	Scikit-Learn	Backtesting evaluation (MAE, RMSE, MAPE metrics)

3. Machine Learning Methodology (ARIMA / SARIMA)

The system employs **ARIMA (AutoRegressive Integrated Moving Average)** time-series modeling:
• **p (AutoRegressive Order)**: Uses linear combination of past sales values to predict future values.
• **d (Differencing Degree)**: Subtracts consecutive values to make non-stationary trend data stationary.
• **q (Moving Average Order)**: Uses past forecast errors in a regression-like model to smooth unexpected noise.

Automatic Model Tuning (Auto-ARIMA): The engine grid-searches parameter combinations *(p, d, q)* and selects the optimal parameters by minimizing the **AIC (Akaike Information Criterion)** value.

4. Model Evaluation & Accuracy Metrics

Metric Name	Mathematical Formula	Business Meaning
MAE (Mean Absolute Error)	1/n ∑ |Actual - Predicted|	Average absolute dollar/rupee error per prediction.

RMSE (Root Mean Squared Error)	$\sqrt{(1/n \sum (\text{Actual} - \text{Predicted})^2)}$	Penalizes large forecast errors heavily.
MAPE (Mean Absolute % Error)	$100\%/n \sum \lvert(\text{Actual} - \text{Pred})/\text{Actual}\rvert$	Percentage deviation relative to actual sales.
Accuracy Score (%)	max(0, 100% - MAPE)	Overall percentage model fit accuracy.

5. Key Application Features

1. **CSV Data Drag & Drop**: Auto-detects Date and Revenue columns with live preview snippet.
2. **Resampling & Data Health**: Handles missing daily gaps via linear interpolation; resamples Daily, Weekly, Monthly.
3. **Multi-Horizon Forecasting**: Project sales 7, 30, 90, or 180 days into the future.
4. **95% Confidence Bounds**: Displays upper and lower risk bounds for inventory planning.
5. **Indian Rupee (■) Currency Support**: Formatted using Indian numbering locale (e.g. ■1,45,719.30).
6. **CSV Forecast Report Export**: Download generated predictions for business spreadsheet reporting.

6. Codebase File Structure

File / Folder	Functionality
app.py	Flask REST API routes (/api/upload, /api/forecast, /api/export-csv)
model.py	ARIMA model fitting, Auto-AIC tuning, backtest evaluation & confidence bounds
forecast.py	Data preprocessing, column auto-detection, date parsing, linear interpolation
templates/index.html	Glassmorphism dark theme dashboard user interface template
static/js/main.js	Chart.js integration, AJAX API handler, dynamic table & currency formatting
static/css/style.css	CSS glassmorphism styling, glowing cards, and responsive layout

7. Presentation & Q&A; Guide for your Professor/Sir

Q1: What is the main objective of your project?

Answer: To develop a web application that takes historical sales CSV data and predicts future sales using Time Series ARIMA modeling to prevent stockouts and overstocking.

Q2: Why did you choose ARIMA over traditional linear regression?

Answer: Standard linear regression assumes independent observations. Time series sales data has autocorrelation (today's sales depend on yesterday's sales) and trends. ARIMA specifically accounts for lagged values (AR) and lagged forecast errors (MA).

Q3: How do you handle missing dates or gaps in sales data?

Answer: In `forecast.py`, we resample the series to a fixed daily/weekly frequency and use linear interpolation (`interpolate(method='linear')`) followed by forward/backward fill.

Q4: How do you evaluate the model's accuracy?

Answer: We perform a sequential 80-20 train-test split. We fit ARIMA on the training data, predict the test period, and compare against actual values using MAE, RMSE, and MAPE metrics.

Q5: What is the 95% Confidence Interval in your chart?

Answer: It represents the statistical upper and lower uncertainty range within which future actual sales are 95% likely to fall, providing a safety threshold for inventory stock planning.